Zero-spare MP-first proof search at 215 states

A new composition with seven permanently protected theorem heads

Research contribution for Vladimir Reshetnikov's ProveIt project

21 October 2026 UTC

Abstract

We construct a materially different 215-working-state, binary, single-tape machine started on an everywhere blank tape. Its command language is the newly published MP-first transfer language; its physical allocator uses zero-valued spare blocks instead of the parent's unit-valued blocks. Neither feature is claimed as newly invented in isolation. Their combination has a different transition graph and a different initialization proof. The exact 73,172-step blank prefix reaches a matched interior axiom-emission frame with an initially unsafe cache. Completion of that axiom leaves eight theorem heads above a suffix whose theoremhood is not assumed. A Cantor-diagonal credit theorem for the MP-first enumeration bounds all future conservative debt by one, permanently protecting the suffix below at least seven heads.

The machine is accompanied by its full transition table, source, construction recipe, exact native/dense startup replay, and arbitrary-natural primitive-runtime certificate. The numerical calculus and its ZFC calibration are inherited with explicit attribution. This is a new composition and ordinary mathematical proof, not a claim of 214 states, minimum size, unconditional nonhalting, or a newly compiled proof-assistant endpoint for these exact bytes.

Contents

1 Construction and comparison baseline **1**

2 The command calculus and the protected tail **3**

3 Completeness and the scope of the equivalence **4**

4 Verification, reproducibility, and novelty **5**

5 A rejected 215-state experiment and a reusable new lemma **6**

A Complete canonical transition table **8**

1 Construction and comparison baseline

The inspected repository snapshot is `f5b2db607f8`, including batch 200, commit `0c8ee615db0`. That batch explicitly distinguishes materially different command languages, enumeration, spare representations, and startup calibrations from mere state renamings, source reassociations, or speed-ups. We use that stricter screen here.

The parent is the MP-first transfer implementation published in the *T215 Transfer* packet. Its source is preserved exactly: 4,775 primitive occurrences, 201 distinct source subtrees, maximum source depth 44, and maximum zero run 6. The only physical row change is

$$\text{native state 198 on symbol 0:} \qquad 1L40 \longmapsto 0L40.$$

The initial state is changed from native 0 to native 74. The new table still has 215 working states and 430 transitions. The designated native dispatcher is 84 and the halt label is 254. A supplied bijection puts the machine in dense form with initial 0, working labels 0 through 214, and halt 215.

In the runtime representation, a spare value n occupies the unary block $B(n) = 01^{n+1}$. Every source increment now appends $B(0) = 01$ rather than $B(1) = 011$. Arbitrary finite nonempty lists of natural spare values are allowed in the simulation invariant. The one changed write is therefore a change to the runtime storage mechanism, not just a transient bootstrap adjustment.

Relative to the previously delivered zero-spare transfer machine, MP and S exchange command tags. That changes both the physical dispatch graph and the candidate words being enumerated. In particular the old zero-spare transfer proof needed conservative credit 3, whereas this source needs credit 1. Relative to the unit-spare MP-first machine, spare allocation and blank initialization both change. The new composition is not presented as the invention of either constituent technique.

1.1 Exact file identities

The native table is identified by SHA-256

`0f201336eda34691f2892c217431bdd2524e813d805645cb0b4a23e58168f7d9`.

The dense table is identified by

`7207d21fad2859064dd145511ebd28866bd227ef54ec6ed6e1b0c3960b452a86`.

The unchanged source is identified by

`8ddb942f0f1d8e2e3b12ab7217c7232eb553cd288fca38dbf9b0eded73893dd5`.

The manifest and construction recipe bind the source, both tables, the map between labels, and all evidence. Hashes identify artifacts; they do not replace the mathematical arguments.

1.2 The complete blank-tape connection

Literal native/dense lockstep execution reaches dispatcher 84 after exactly 73,172 transitions, at head coordinate -2, with gap 0 and source address

`11010011000110110110` (ordinal 119).

In logical register order $(N, T, W, A, D, H, X, Y, L)$, the frame has values

$$(0, 6, 0, 0, 1, 1, 10, 1, 0).$$

Its 65 spare blocks are all 0 except the third, which is 1. Its complete tape has 107 one-cells. The audit reconstructs *every* primary and spare block, the source-address region, header, and zero exterior; finding dispatcher 84 alone would not establish a valid source frame.

A separate source-control-flow replay starts at AX entry ordinal 25 with values $(0, 0, 0, 0, 1, 1, 0, 0, 0)$. After 177 primitive steps it reaches exactly ordinal 119 and the register vector above. This identifies the physical initialization as a genuine interior AX computation. It does not assert that cache 1 is a theorem, nor that the address is the source root. The remaining all-natural AX suffix is handled by the source theorem and full physical simulation.

2 The command calculus and the protected tail

The theory being searched is the inherited numerical calculus, whose theorem codes are natural numbers. Code 0 is truth and code 1 is falsity. Its calibration with ZFC is an inherited theorem, not an inference from running the machine. Arbitrary natural numbers, including codes without a first-order reading, are permitted as parameters. This flexibility does not permit an unproved code to be used as a premise.

Let $P(x, y) = (x + y)(x + y + 1)/2 + y$, and let $f(n)$ be the second component of the inverse Cantor pairing. The native constructors used below are $I(a, b) = 4P(a, b)$ and $Q_i(a) = 4P(i, a) + 2$. A stack with explicit heads $x_1, \ldots, x_k$ and tail J means the iterated numerical code $P(x_1, P(x_2, \ldots, P(x_k, J)))$. This is a representation with a specified list of heads, not a claim about a unique list length. In particular $P(0, 0) = 0$ creates no difficulty: the identities for unpairing still recover each designated wrapper when it is consumed.

At a command boundary the work registers are (t, w, d, h), consisting of the current theorem, stack, additive parameter counter, and cached theorem. The command alphabet is

$$\mathrm{MP}(x) : [0, x], \qquad S : [1], \qquad \mathrm{AX}(c; x, y) : [c + 2, y, x].$$

The transfer command is the literal composite old-DROP; old-NIP; old-NIP. If $w = P(z, v)$ it returns (h, v, d, z), unless the incoming $h = 1$ or $t = 1$, in which case it halts. Both tests matter. Its temporary internal push means that the internal implementation does not reach farther into the old stack than its net one-head removal.

Numerical $\mathrm{MP}(x)$ sets $y = d + x$ and tests whether $h = I(t, y)$. Success returns $(y, P(0, w), y, 0)$; failure returns $(0, w, y, 0)$. It does not remove an old stack head. AX emits the accepted logical and set-theoretic axiom codes, including the dominated eighth logical row, with parameters $U = d$, $A = d + x$, $B = h + y$, $C = c$. Thus $U \leq A$ holds for all natural inputs. It resets d, h and prepends ten heads, of which the generalization of the incoming cache and the saved incoming top require care when these inputs are untrusted.

2.1 The dirty initialization is not a theorem-valued state

The literal blank prefix ends inside AX. Its matching virtual source entry has

$$(N, t, w, A, d, h, X, Y, L) = (0, 0, 0, 0, 1, 1, 0, 0, 0).$$

The remaining encoded fields are zero. Completion therefore uses $U = A = B = 1$, $C = 0$. The returned top is the enabled set-theoretic bundle, and the new cache is truth. The resulting explicit stack is

$$[\rho_1, \ldots, \rho_7, L(1, 1, 0, 1), Q_1(1), 0; 0].$$

The first eight heads are theorems by the inherited axiom contracts. We make *no* theoremhood assertion about $Q_1(1)$. Its code is 18, and the two-head suffix is the arbitrary numerical tail

$$J = P(18, P(0, 0)) = 171.$$

It remains to prove that this suffix is never exposed by the actual search. An unjustified claim that the whole startup stack consists of theorems would invalidate the construction.

2.2 An infinite credit theorem, not a finite simulation

Assign conservative weights $\mathrm{wt}(\mathrm{MP}) = 0$, $\mathrm{wt}(S) = -1$, $\mathrm{wt}(\mathrm{AX}) = 10$. Successful MP can only improve the resulting bound. Let $W(n)$ be the syntactic command word of a freshly fetched

candidate, continuing syntactically even if execution would halt. Put $R(0) = \epsilon$ and $R(r) = W(r-1)$ for $r > 0$. If $n = P(a, y)$, the actual decoder satisfies

$$W(n) = \begin{cases} \mathrm{MP}; R(f(y)), & a = 0, \\ S; R(y), & a = 1, \\ \mathrm{AX}; R(f(f(y))), & a \geq 2. \end{cases}$$

Every positive recursive argument decreases; every prefix of $R(r)$ has weight at least $-r$. The initial words are $W(0) = \mathrm{MP}$, $W(1) = S$, and $W(2) = \mathrm{MP}$, so the cumulative minimum through these candidates is -1.

Here is the unbounded diagonal argument underlying the inherited credit theorem. Write $b(n) = \mathrm{wt}(W(n))$, $B(k) = \sum_{n<k} b(n)$, and $g(0) = 0$, $g(r) = b(r-1)$ for $r > 0$. For $q(r) = g(r) + 1$, for $k \geq 1$ the first k terms sum to $k + B(k-1)$. The induction invariant $B(j) + j \geq 0$ therefore makes all these prefixes nonnegative. The sequence $f(0), f(1), \ldots$ is the concatenation $[0], [0, 1], [0, 1, 2], \ldots$. Consequently pulling a nonnegative-prefix sequence back by f, once or twice, preserves nonnegative prefix sums.

Maintain also $B(T_s) \geq -1$, where $T_s = s(s+1)/2$; the three initial words establish this at $s = 2$. On Cantor diagonal $s \geq 2$, the first $s-1$ candidates start with AX, followed by one S candidate and one MP candidate. The first k AX candidates have weight at least $9k$. Inside the AX candidate at position y, its initial $+10$ and its continuation bound give relative weight at least $8y + 10$. After all AX candidates the balance is at least $9(s-1)$. During the S candidate it is at least $8s - 9$, and during the final MP candidate at least $7s - 9 > 0$. All recursive information used here comes from earlier diagonals. Thus every diagonal from $s = 2$ is prefix-safe relative to its beginning, preserving $B(T_s) \geq -1$ and $B(j) + j \geq 0$ and completing the induction.

Theorem 2.1 (One-head conservative debt)**.** *Every prefix of the infinite syntactic search from candidate 0 has weight at least -1. The first S attains -1. For the semantic conclusion, start at a source root with next candidate 0, empty proof pointer, normalized selector and scratch registers, theorem-valued top and cache, and eight explicit theorem heads above an arbitrary tail. The additive counter may be any natural number. Then at least seven such heads remain at every command boundary; the arbitrary tail is never popped into an active theorem register.*

The conclusion uses explicit wrapper counts, not numerical stack size. Before any S, at least one protected theorem head is available. Each possible halt therefore tests a theorem-valued falsity code, including the extra top test inside the composite S. This proves soundness without any assumption about the buried suffix.

3 Completeness and the scope of the equivalence

The inherited transfer compiler has a strong block contract over an *arbitrary retained stack*: with $d = h = 0$ and normalized parser scratches, but arbitrary retained t, w, it constructs each derivable code F as $(F, w, 0, 0)$, or halts earlier. Its cleanup and command composition proofs distinguish the actual two-guard S from the single-guard abstraction. Early halting is permitted in the completeness contract; soundness for the actual run is supplied separately by the protected-tail invariant.

The MP-first change is a translation of command tags and their literal field encoding, not a change of the numerical theory. An initial AX reset supplies the zero counter/cache precondition independently of the incoming natural work tuple. The cofinal padding proof supplies successful encodings above every finite candidate cutoff when falsity is derivable. Each preceding nonhalting candidate terminates, because its arithmetic operations terminate and its positive continuation

code strictly decreases. In the present source the scheduler enumerates every candidate successively. Hence a finite numerical refutation eventually causes a halt. No bounded search experiment establishes this assertion; the strong block and cofinality proofs are retained in the packet.

Combining that completeness theorem, protected-tail soundness, the exact blank prefix, and the arbitrary-natural physical simulation yields

$$\begin{aligned} \text{Blank halting} &\iff \text{the numerical calculus derives } 1 \\ &\iff \neg\,\mathrm{Con}(\mathrm{ZFC}). \end{aligned}$$

Equivalently its nonhalting expresses syntactic ZFC consistency. These are ordinary mathematical arguments intended to be formalizable in ZFC, with executable symbolic certificates for the physical runtime. They are not a new proof-assistant kernel endpoint for this table, do not prove unconditional nonhalting, and do not prove minimality.

4 Verification, reproducibility, and novelty

The proof obligations are deliberately separated.

1. **Literal construction.** Rebuild the table from the pinned parent, its one changed row, and its designated start; check all working states, outgoing rows, targets, and dense renaming.
2. **Blank connection.** Replay every transition of the finite initialization in independent sparse-tape and dense/native forms; reconstruct the entire endpoint frame and compare the source AX prefix.
3. **Unbounded physical simulation.** For each of the 6,209 guarded source primitive cases, use symbolic induction over arbitrary unary runs and block lists, with arbitrary natural primary values, gap, and nonempty finite spare lists. This is not a bounded test of small register values.
4. **Source soundness and completeness.** Use the exact command contracts, protected-tail theorem, strong compiler, and cofinal encoding. None follows merely from a primitive census or a successful blank prefix.
5. **Theory calibration.** Use the explicitly inherited numerical-calculus equivalence with ZFC consistency.

The fresh runtime proof passes all 6,209 guarded cases, with 1,610,781 symbolic rule applications, minimum affine duration constant 10, and coverage of all 430 physical rows. The frozen receipt binds these counts to the exact source and table hashes. The portable verifier rederives the runtime proof when its full-runtime option is selected. The default verifies the exact artifact and stored evidence, and must not be described as a fresh rederivation of all runtime cases.

The independent comparison uses 77 distinct 215-state tables representing 44 rooted transition-graph classes. It covers all 28 table paths in the current `candidate-k215` subtree at the pinned repository snapshot, the relevant new incoming archives, and the retained historical comparison tables. The new machine has no rooted state-renaming match, including after a global tape reflection. A frozen corpus and offline checker make this bounded statement reproducible. It is not a claim to have compared every Turing machine or every unpublished construction.

4.1 Proof-assistant status is table-specific

Upstream commit `613f9cf28ac` publishes compiled Lean and Rocq endpoint evidence for the original accepted literal K215. That result must not be described as still pending. It also must not be transferred automatically to this changed table. The original dense-table SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

The present report does not independently replay those kernel builds and does not claim a new exact-byte Lean or Rocq endpoint. The current composition is supported by ordinary mathematical proofs and executable symbolic checks. Internal encoded PA or ZFC derivations are separate from external proof-assistant checking.

4.2 References and provenance

The complete parent packet, including the ProveIt and Andrew James Wade attribution and license notices, source-language proofs, numerical-calculus references, and runtime engine, is retained. The numerical-calculus dependencies are identified in `parent/reference/accepted_k215_proof.md` and `parent/reference/dominated_rho8_proof.md`. In particular the command-completeness dependencies are `parent/reference/transfer-parent/proofs/transfer_compiler.md`, `composite_transfer_review.md` in that directory, and `parent/proofs/permuted_credit.md`. These are the three-command transfer proofs, not the older four-command compiler with a different AX tag offset.

The current upstream intake is available at `https://github.com/VladimirReshetnikov/ProveIt/commit/0c8ee615db0f3f1e0e1083b7b2a3d085e12c654a`. The original literal K215 kernel evidence is linked from `https://github.com/VladimirReshetnikov/ProveIt/blob/613f9cf28ac36eaa929f7ef909b0eb7905facc5d/Computability/BusyBeaver/Research/ZFC/k225-formal/verification/k215/README.md`. These historical pins are not claims that no later repository changes exist.

5 A rejected 215-state experiment and a reusable new lemma

The latest archives include a separate scratch-biased short-transfer experiment. Its 6,152-case physical certificate is not a consistency calibration. Our audit finds a stronger problem than the stated open cofinality question: its literal program does not implement its advertised scheduler.

The original return was `CC3` at address `10011`. Replacing that leaf by the pair `(CI6,CC3)` moves the continue instruction to `100111`. A structured continue counts right ancestors; the newly introduced right ancestor changes its target. Two independently implemented control-flow resolvers agree over all 4,733 source occurrences. At ordinal 4697, this `CC3` returns to the selector at ordinal 23, *not* the scheduler at ordinal 0. A direct semantic repair at that address would require `CC4`, whose physical realization is a separate obligation.

After a normal AX or MP, the selector, cache, and arithmetic scratches are zero. The extra `CI6` makes scratch $X = 1$. The wrong return therefore enters the short-transfer branch, clears the active top, and executes its cache-copy restoration phase with this preexisting scratch unit. That phase creates $H = 1$. The subsequent NIP test reaches the halt instruction, independently of the stack and other retained natural data. If the incoming top is t, the source reaches CH after $2t + 15$ nonhalting primitives; the retained $t = 42$ witness has 99 such steps, followed by CH itself. We do not claim a measured whole-blank binary halting time. The archived startup enters an AX suffix, whose totality and physical realization give the mathematical connection to this unavoidable false halt.

The portable rejection audit retains the exact source and table, two control-flow reconstructions, the complete local witness, and its provenance. This illustrates why a full arbitrary-natural primitive certificate can be correct while the overall program is wrong. The rejected bytes are not included as a successful second consistency machine.

5.1 Adjacent successful candidates for the intended biased schedule

A useful new encoding lemma survives independently of those rejected bytes. Consider the *intended* stateful schedule with S tagged 0, MP tagged 1, AX tagged at least 2, and bias 0 after S versus bias 1 after MP or AX. At a positive residual r, it reads $r - 1 + b$, where b is the bias; a fresh candidate is read without bias. Consequently, if $n = P(a, y)$, its fresh word starts with S and continues at $y - 1$ when $a = 0, y > 0$; starts with MP and continues at $f(y)$ when $a = 1, f(y) > 0$; or starts with AX and continues at $f(f(y))$ when $a \geq 2$ and that residual is positive. Zero residual terminates the word. Each positive recursive argument is smaller than n.

Assume $r > 0$ is a fresh code for a suffix that halts from *every* natural work tuple. For any $v \geq 1$, set

$$u = P(1, r), \qquad y = P(v, u), \qquad n = P(2, y).$$

The elementary identities

$$n + 1 = P(1, y + 1), \quad y + 1 = P(v - 1, u + 1), \quad u + 1 = P(0, r + 1)$$

show that candidate n executes $\mathrm{AX}(0; 1, v)$ followed by the universal suffix $W(r)$, while candidate $n + 1$ executes $\mathrm{MP}(v - 1)$, then S, then the same suffix. Both halt from every initial tuple, either earlier or in that suffix. As v grows, these adjacent successful pairs are cofinal. An increasing enumeration with jumps of at most two cannot skip both members of any such pair above its starting point.

This is a conditional mathematical theorem, not a calibration of the rejected source. To use it in a repaired machine still requires a literal parser correspondence, a stateful strong wire compiler, and a soundness invariant. The attached note states these premises explicitly. No repaired 214- or 215-state machine is asserted on the strength of this lemma alone.

A Complete canonical transition table

The working labels are $0, \ldots, 214$. The initial state is 0; halt H denotes label 215 and has no outgoing transitions. The initial head is at coordinate 0 on an all-zero tape. An entry wDq writes w, moves one cell in direction D, and enters q. This is the complete machine, not a preinitialized simulator.

q	0	1	q	0	1	q	0	1
0	1R1	1R64	72	0R56	1L78	144	0R83	1R87
1	1R91	1R1	73	0R79	1R80	145	0R156	1R13
2	0L2	1L3	74	0R74	1R81	146	0R82	1R141
3	0L4	1L3	75	0L143	1L139	147	0R161	1R165
4	0L6	1L3	76	0R84	1R85	148	0L67	1L139
5	1R7	0L5	77	0R88	1L131	149	0R146	1R199
6	0L10	1L3	78	0L107	1R57	150	0R156	1R113
7	0R7	0R11	79	0R49	0R34	151	0R96	1R96
8	0R83	1R187	80	0R90	0L42	152	0R49	0R33
9	0R15	1R14	81	0R92	1R174	153	0R69	1R200
10	0L16	1L3	82	0R95	1R13	154	0L207	1R95
11	0L17	0R11	83	0R87	1R96	155	0R165	1R176
12	0L27	1R128	84	0R203	1R22	156	0R20	1R167
13	0R20	1R21	85	0R115	1R99	157	0R170	1R162
14	0L88	1L32	86	0R148	1R112	158	0R156	1R95
15	0L12	0R23	87	0R101	1R102	159	0R124	1R161
16	0L24	1L3	88	0R79	1R78	160	0R96	1R159
17	1L2	0L2	89	0R100	1R171	161	0L8	1R140
18	0R25	0R33	90	0R104	1R105	162	0L108	1R170
19	0R198	1R197	91	0R66	0R1	163	0R144	1R169
20	0R29	1R30	92	0R28	1R106	164	0R193	1R29
21	0R31	1R32	93	0R179	0L204	165	0R158	1R141
22	0R25	0R60	94	1L109	1L94	166	0R175	1R175
23	0R34	1R23	95	0R20	1R110	167	0R178	1R121
24	0L35	1L3	96	0R61	1R61	168	0R143	1R162
25	0L36	0R1	97	0R164	1R154	169	0R75	1R61
26	0R37	1R26	98	0R67	1R186	170	0R143	1R87
27	0R127	1R38	99	0R104	0L204	171	0R103	1R95
28	0R39	0L40	100	0R116	1R117	172	0R138	1R83
29	0R41	1R42	101	0R72	1R59	173	0R172	1R75
30	0R43	1R44	102	0R58	1R118	174	0R107	1R214
31	0R45	1R46	103	0R22	1R119	175	0R140	1L147
32	0R22	1R47	104	0R22	0L11	176	0R96	1R52
33	0R48	1R33	105	0R62	1R42	177	0R165	1R52
34	0R33	1R34	106	0R120	1R9	178	0R18	1L130
35	0L50	1L3	107	0R100	1R121	179	0R76	1R115
36	1R36	1L2	108	0R148	1R183	180	0R105	1R123
37	0R51	1R37	109	0L204	0L94	181	0L40	1L109
38	0R14	1R53	110	0R123	1R103	182	0R168	1R156
39	0L12	0R37	111	0R206	1R180	183	0R148	1R114
40	0L5	1L54	112	0R162	1R178	184	0R12	1R212
41	0R25	0R26	113	0R146	1R144	185	0R189	1R98
42	0R14	0L17	114	0R157	1R197	186	0R144	1R195
43	0R56	1R57	115	0L130	0L2	187	0R154	1R188
44	0R58	1R59	116	0R39	1R73	188	0R172	1R182
45	0R25	0R55	117	0R62	1R131	189	0R65	1R8
46	0R61	1R62	118	0R62	1R132	190	0R148	1R126
47	0R63	0L17	119	0R71	0L17	191	0R153	1R111
48	0R26	1R48	120	0R15	1R131	192	0R150	1R122
49	0L36	0R64	121	0R22	1R133	193	0R89	1R87

q	0	1	q	0	1	q	0	1
50	0L74	1L3	122	0R145	1R175	194	0R184	0L204
51	0R17	1R51	123	0R135	1L21	195	0R202	1R108
52	0R145	1R141	124	0R129	1R177	196	0R148	1R166
53	0R68	1R69	125	0R201	1R168	197	0R196	1R157
54	0L40	1L54	126	0R75	1R173	198	0R138	1R136
55	0R23	1R55	127	0R18	1R194	199	0R192	1R19
56	0L12	0R51	128	0R75	1R134	200	0R87	1R205
57	0R61	0L17	129	0R13	1R139	201	0R160	1R190
58	0R70	1R71	130	0R31	1R70	202	0R125	1R149
59	0R62	1R72	131	0R39	1L123	203	0R45	1R99
60	0R55	1R60	132	0R151	0L17	204	0L5	0L204
61	0R49	0R26	133	0R152	0L17	205	0R193	1R138
62	0L29	1L73	134	0R158	1R137	206	0R161	1R207
63	0L116	1L31	135	0R25	0R48	207	0R208	1R179
64	0R0	1R64	136	0R138	1R177	208	0R141	1R209
65	0R96	1R148	137	0R191	1R93	209	0R142	1R210
66	1R181	1L54	138	0R95	1R156	210	0R211	1R86
67	0R142	1R143	139	0R172	1R163	211	0R185	1R162
68	0R76	1RH	140	0R13	1R156	212	0R213	1R38
69	0L193	1R32	141	0R87	1L163	213	0R13	1R38
70	0R41	1R63	142	0R87	1R155	214	0R127	1R97
71	0L88	1R77	143	0L150	1L160			